ITAI 1371 ? Module 10

Individual Student Submission

Aleander Debusk

Unsupervised Learning Lab

Module 10 Lab - Unsupervised Learning**Objective:** To understand and apply two common **unsupervised learning** techniques: **K-Means Clustering** for finding groups in data and **Principal Component Analysis (PCA)** for dimensionality reduction.**In this lab, you will write the code to perform clustering and PCA, and you will interpret the results.**

Part 1: Supervised vs. Unsupervised LearningSo far, every model we have built has been a **supervised learning** model. This means we had **labeled data**—we had both the features (X) and the correct answer, or target (y). The goal was to learn a mapping from X to y.In **unsupervised learning**, we only have the features (X). There is **no target variable**. The goal is to discover interesting structures or patterns hidden in the data on its own.Two major types of unsupervised learning are:* **Clustering:** Automatically grouping similar data points together.* **Dimensionality Reduction:** Compressing the data by reducing the number of features while trying to preserve as much information as possible.

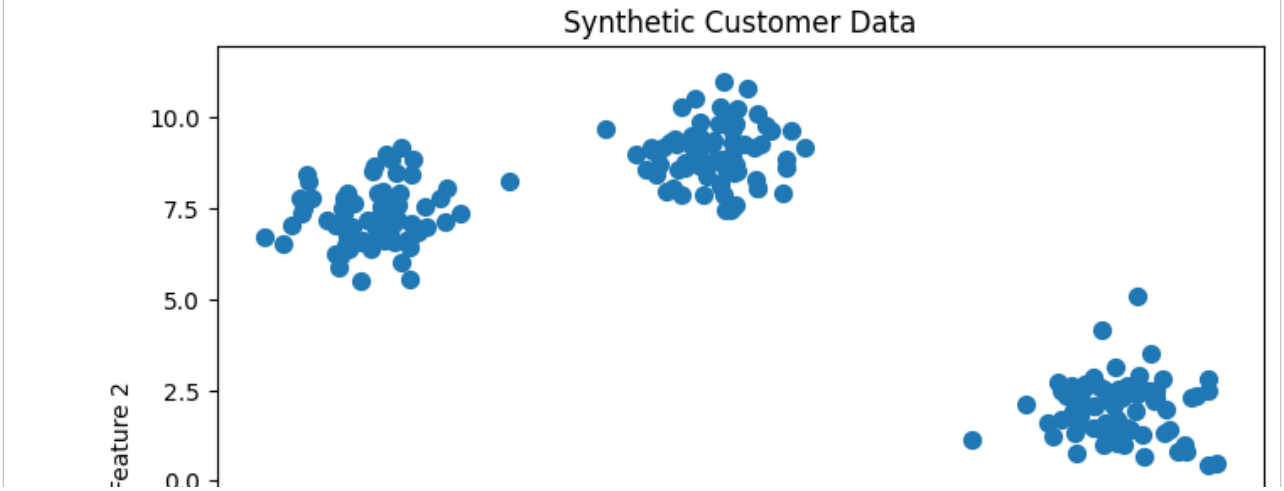
Part 2: K-Means Clustering**Concept:** K-Means is an

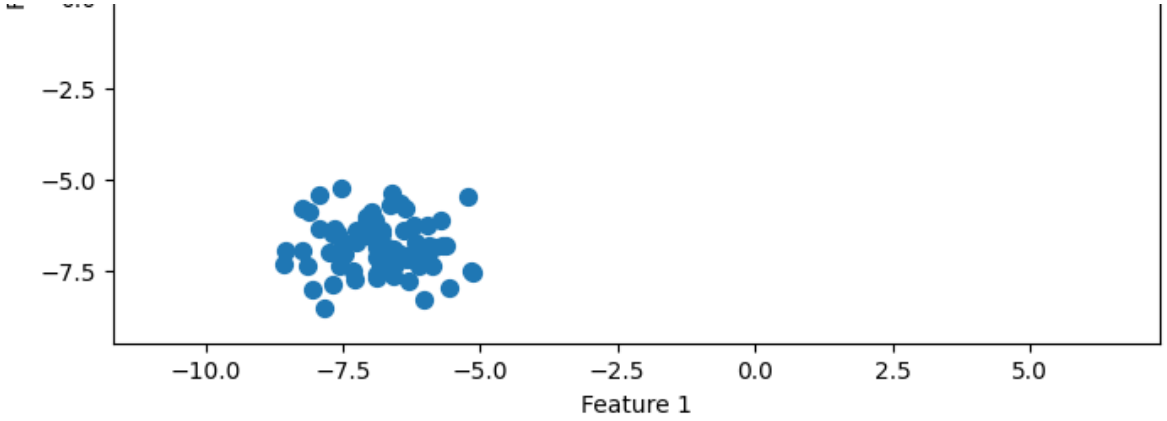
algorithm that finds a pre-defined number of clusters (`k`) in a dataset. It works by:1. Randomly placing `k` centroids (the center of a cluster) in the data.2. **Assignment Step:** Assigning each data point to its nearest centroid.3. **Update Step:** Moving each centroid to the average position of all the points assigned to it.4. Repeating steps 2 and 3 until the centroids stop moving.**Problem:** We will use a synthetic dataset of customer spending habits to segment them into groups.

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

# Generate synthetic data
X, y = make_blobs(
    n_samples=300,
    centers=4,
    cluster_std=0.8,
    random_state=42
)

# Visualize the data
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], s=50)
plt.title("Synthetic Customer Data")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```





Task 1: Find the Optimal Number of Clusters (The Elbow Method)**Concept:** How do we know what `k` should be? The **Elbow Method** helps us decide. We run K-Means for a range of `k` values and for each `k`, we calculate the **inertia** (the sum of squared distances of samples to their closest cluster center). We then plot the inertia for each `k`. The "elbow" of the curve—the point where the inertia starts to decrease much more slowly—is a good estimate for the optimal `k`.**Your Task:** Calculate and plot the inertia for `k` values from 1 to 10.

```
from sklearn.cluster import KMeans

# --- ENTER YOUR CODE HERE ---

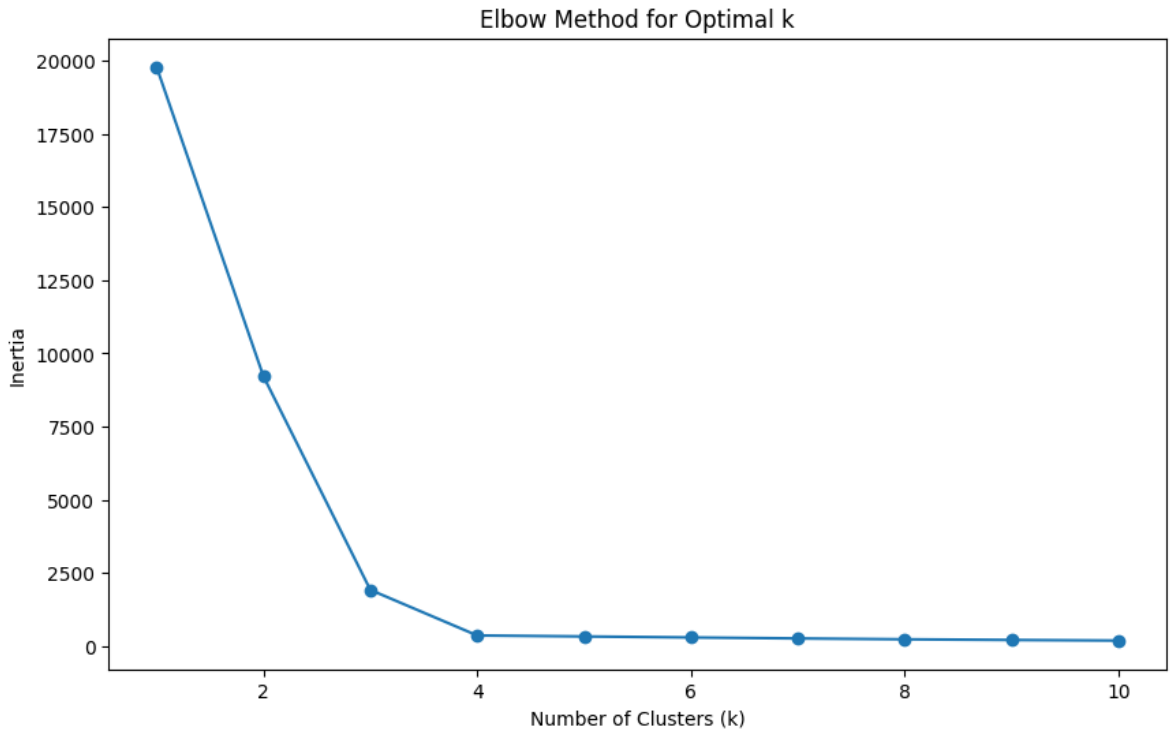
# 1. Create an empty list to store the inertia values
inertia = []

# 2. Loop through k values from 1 to 10
for k in range(1, 11):

    # 3. Create and fit a KMeans model with n_clusters=k
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X)

    # 4. Append the model's inertia_ to your list
    inertia.append(kmeans.inertia_)
```

```
# 5. Plot the results
plt.figure(figsize=(10, 6))
plt.plot(range(1, 11), inertia, marker='o')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.show()
```



Task 2: Perform K-Means Clustering and VisualizeFrom the elbow plot, it looks like `k=4` is the optimal number of clusters.**Your Task:** Run K-Means with `n_clusters=4` and create a scatter plot showing the data points colored by their assigned cluster.

```
# --- ENTER YOUR CODE HERE ---

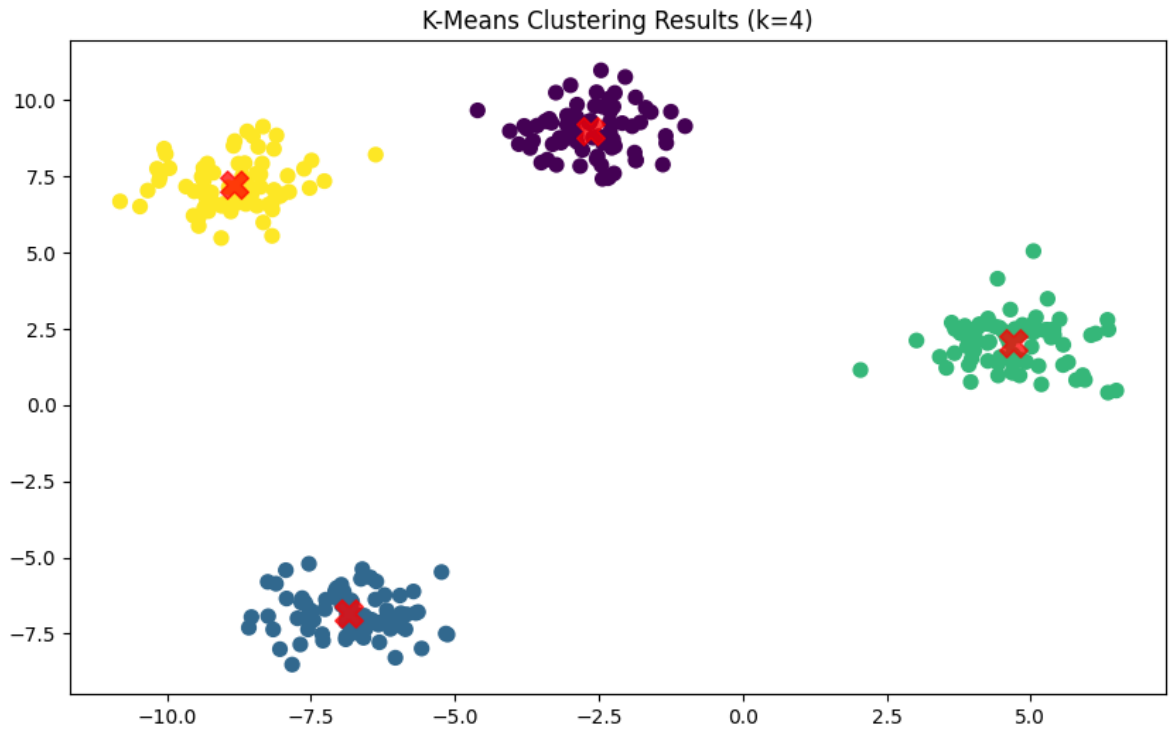
# 1. Create and fit a KMeans model with n_clusters=4
```

```
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
kmeans.fit(X)

# 2. Get the cluster assignments for each data point
labels = kmeans.labels_

# 3. Get the coordinates of the cluster centers
centers = kmeans.cluster_centers_

# 4. Create a scatter plot of the data, colored by the labels
plt.figure(figsize=(10, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap='viridis')
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.75, marker
plt.title('K-Means Clustering Results (k=4)')
plt.show()
```



Part 3: Principal Component Analysis (PCA)**Concept:** PCA is a dimensionality reduction technique. It transforms the data into a new coordinate system of **principal**

data into a new coordinate system of **principal components**. These components are orthogonal (uncorrelated) and are ordered by the amount of variance in the data they explain. By keeping only the first few principal components, we can reduce the number of dimensions in our data while losing as little information as possible.**Use Cases:*** **Visualization:** Reducing high-dimensional data to 2 or 3 dimensions so we can plot it.* **Performance:** Speeding up model training by using fewer features.**Problem:** We will use the Iris dataset, which has 4 features, and reduce it to 2 features so we can visualize it.

Task 3: Apply PCA to the Iris Dataset**Your Task:** Use `PCA` from `sklearn.decomposition` to reduce the 4-dimensional Iris dataset to 2 principal components and visualize the result.

```
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the data
iris = load_iris()
X_iris = iris.data
y_iris = iris.target

# --- ENTER YOUR CODE HERE ---

# 1. Create a PCA instance to reduce to 2 components
pca = PCA(n_components=2)

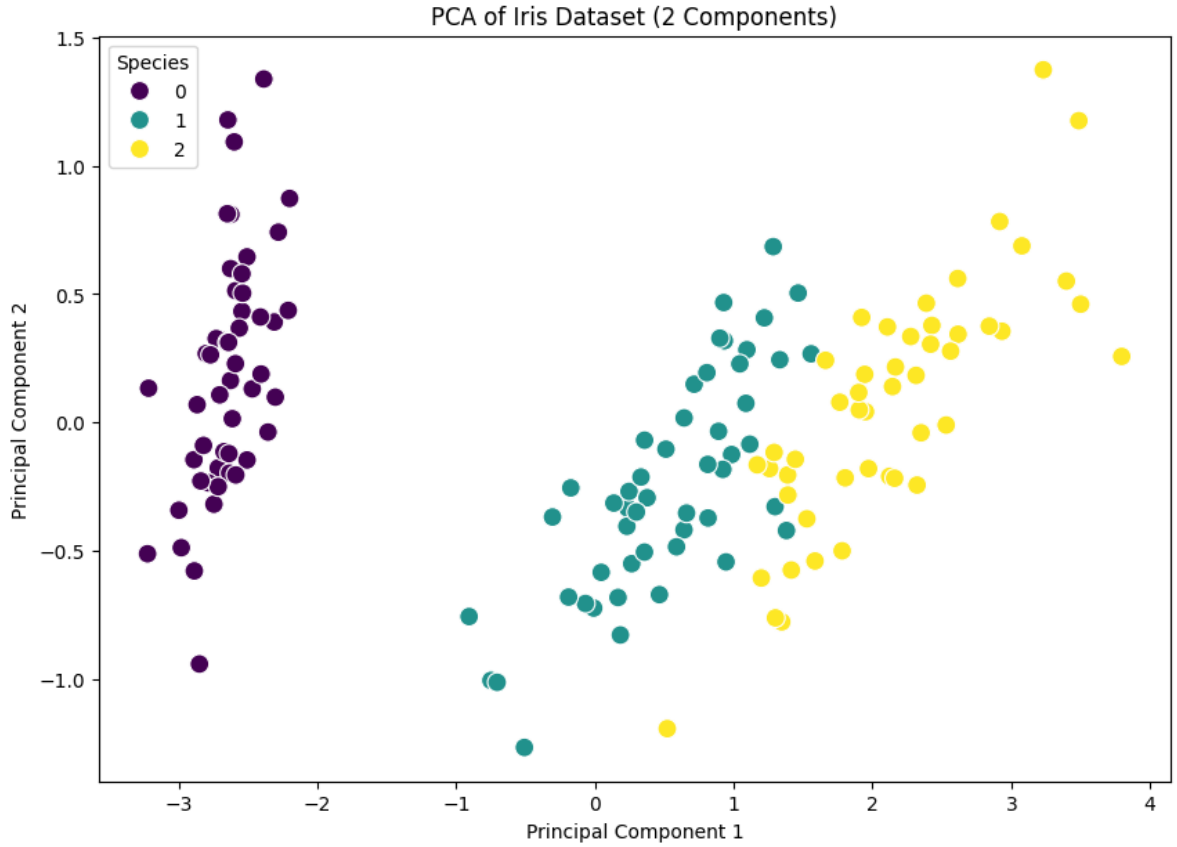
# 2. Fit PCA to the data and transform it
X_pca = pca.fit_transform(X_iris)

# 3. Print the explained variance ratio
print(f"Explained variance by component: {pca.explained_variance_ratio_}")
print(f"Total variance explained by 2 components: {pca.explained_variance_rat
```

```
# 4. Create a DataFrame with the PCA results for plotting
pca_df = pd.DataFrame(
    data=X_pca,
    columns=["Principal Component 1", "Principal Component 2"]
)
pca_df["Species"] = y_iris

# 5. Plot the results
plt.figure(figsize=(10, 7))
sns.scatterplot(
    x="Principal Component 1",
    y="Principal Component 2",
    hue="Species",
    data=pca_df,
    palette='viridis',
    s=100
)
plt.title('PCA of Iris Dataset (2 Components)')
plt.show()
```

```
Explained variance by component: [0.92461872 0.05306648]
Total variance explained by 2 components: 97.77%
```



Close

```
## 📝 Knowledge Check**Instructions:** A
markdown cell.1.  **What is the fundament
unsupervised learning?**2.  **In the K-Me
choose the largest possible `k` to get th
results, the first two principal componer
What does this tell you about the origina
**1. What is the fundamental difference b
learning?

Supervised learning uses labeled data wit
answers (y). Unsupervised learning uses c
find patterns or groups without having cc

2. In the K-Means elbow plot, why don't w
to get the lowest inertia?

Choosing the largest k would create too m
data points into their own groups. This w
necessarily create useful or meaningful c

3. In the PCA results, the first two prir
the variance. What does this tell you abc
dataset?

This means the first two principal compor
information from the original four featur
four dimensions to two while losing very
to visualize and analyze.**
```

📝 Knowledge Check**Instructions:** Answer the following questions in this markdown cell.1. **What is the fundamental difference between supervised and unsupervised learning?**2. **In the K-Means elbow plot, why don't we just choose the largest possible `k` to get the lowest inertia?**3. **In the PCA results, the first two principal components explained over 95% of the variance. What does this tell you about the original 4-dimensional Iris dataset?**

**1. What is the fundamental difference between supervised and unsupervised learning?

learning?

Supervised learning uses labeled data with input features (X) and correct answers (y). Unsupervised learning uses only input features (X) and tries to find patterns or groups without having correct answers.

2. In the K-Means elbow plot, why don't we just choose the largest possible k to get the lowest inertia?

Choosing the largest k would create too many clusters and could put individual data points into their own groups. This would lower inertia but would not necessarily create useful or meaningful clusters.

3. In the PCA results, the first two principal components explained over 95% of the variance. What does this tell you about the original 4-dimensional Iris dataset?

This means the first two principal components preserve over 95% of the information from the original four features. We can reduce the dataset from four dimensions to two while losing very little information, making it easier to visualize and analyze.**

ITAI 1371 ? Module 10

Individual Student Submission

Shareefah Adeola

Unsupervised Learning Lab

Module 10 Lab - Unsupervised Learning

Objective: To understand and apply two common **unsupervised learning** techniques: **K-Means Clustering** for finding groups in data and **Principal Component Analysis (PCA)** for dimensionality reduction.

In this lab, you will write the code to perform clustering and PCA, and you will interpret the results.

Part 1: Supervised vs. Unsupervised Learning

So far, every model we have built has been a **supervised learning** model. This means we had **labeled data**—we had both the features (X) and the correct answer, or target (y). The goal was to learn a mapping from X to y.

In **unsupervised learning**, we only have the features (X). There is **no target variable**. The goal is to discover interesting structures or patterns hidden in the data on its own.

Two major types of unsupervised learning are:

- **Clustering:** Automatically grouping similar data points together.
- **Dimensionality Reduction:** Compressing the data by reducing the number of features while trying to preserve as much information as possible.

Part 2: K-Means Clustering

Concept: K-Means is an algorithm that finds a pre-defined number of clusters (`k`) in a dataset. It works by:

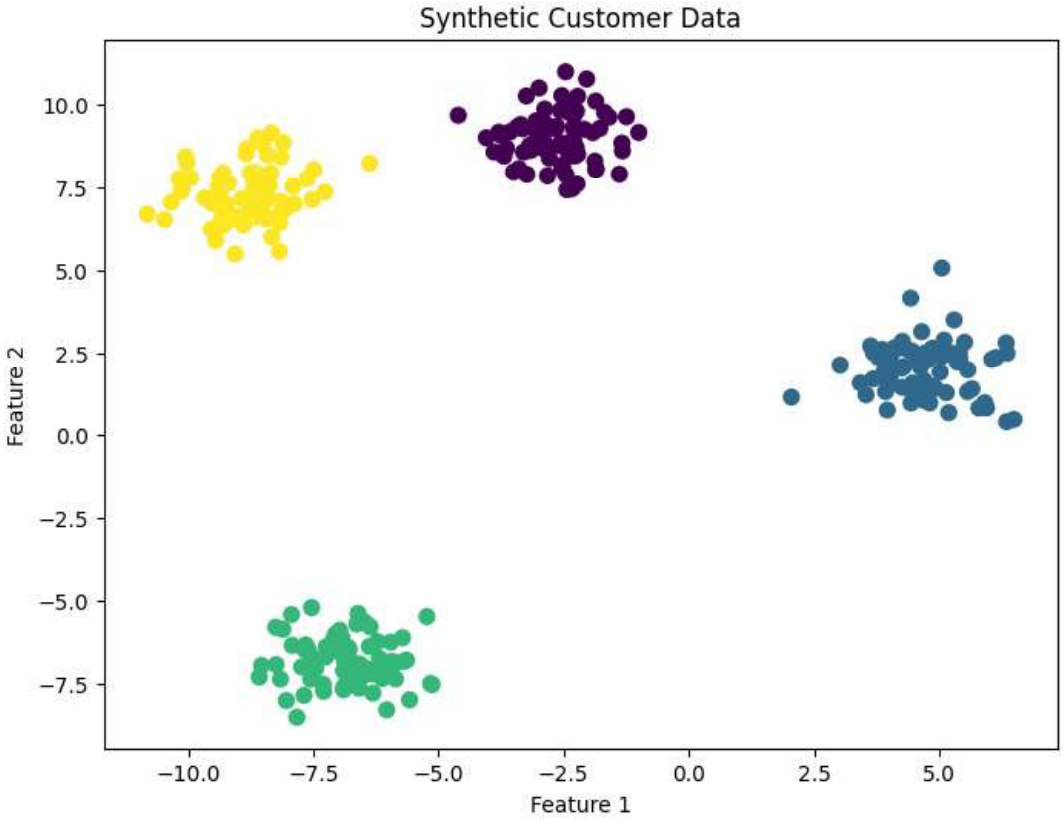
1. Randomly placing `k` centroids (the center of a cluster) in the data.
2. **Assignment Step:** Assigning each data point to its nearest centroid.
3. **Update Step:** Moving each centroid to the average position of all the points assigned to it.
4. Repeating steps 2 and 3 until the centroids stop moving.

Problem: We will use a synthetic dataset of customer spending habits to segment them into groups.

In [1]:
```python
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

# Generate synthetic data
X, y = make_blobs(n_samples=300, centers=4, cluster_std=0.8, random_state=42)

# Visualize the data, coloring points by their generated group
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='viridis')
plt.title("Synthetic Customer Data")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```



Task 1: Find the Optimal Number of Clusters (The Elbow Method)

Concept: How do we know what `k` should be? The **Elbow Method** helps us decide. We run K-Means for a range of `k` values and for each `k`, we calculate the **inertia** (the sum of squared distances of samples to their closest cluster center). We then plot the inertia for each `k`. The "elbow" of the curve—the point where the inertia starts to decrease much more slowly—is a good estimate for the optimal `k`.

Your Task: Calculate and plot the inertia for `k` values from 1 to 10.

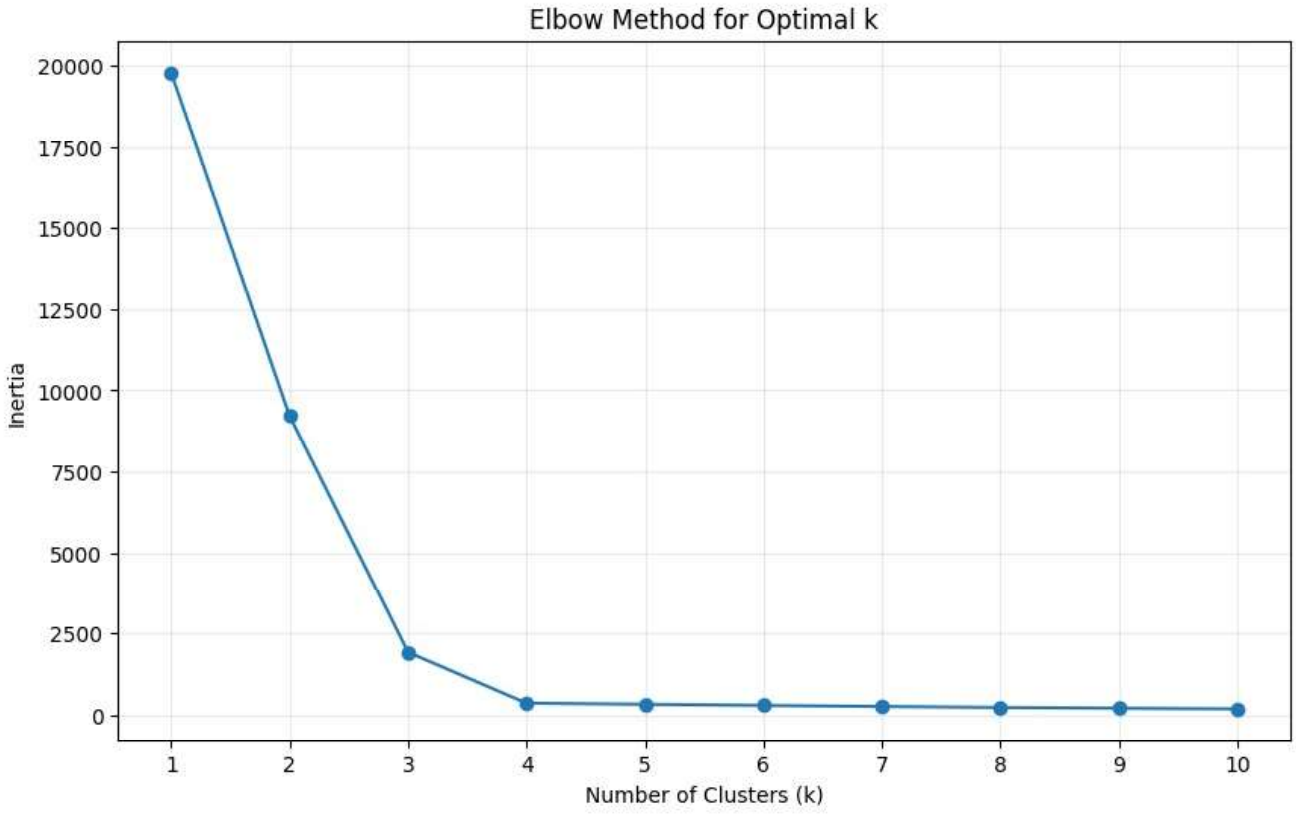
In [2]:
```python
from sklearn.cluster import KMeans

# 1. Create an empty list to store the inertia values
inertia = []

# 2. Loop through k values from 1 to 10
for k in range(1, 11):
    # 3. Create and fit a KMeans model
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X)

    # 4. Append the model's inertia to the list
    inertia.append(kmeans.inertia_)

# 5. Plot the results
plt.figure(figsize=(10, 6))
plt.plot(range(1, 11), inertia, marker='o')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.xticks(range(1, 11))
plt.grid(alpha=0.25)
plt.show()
```



Task 2: Perform K-Means Clustering and Visualize

From the elbow plot, it looks like `k=4` is the optimal number of clusters.

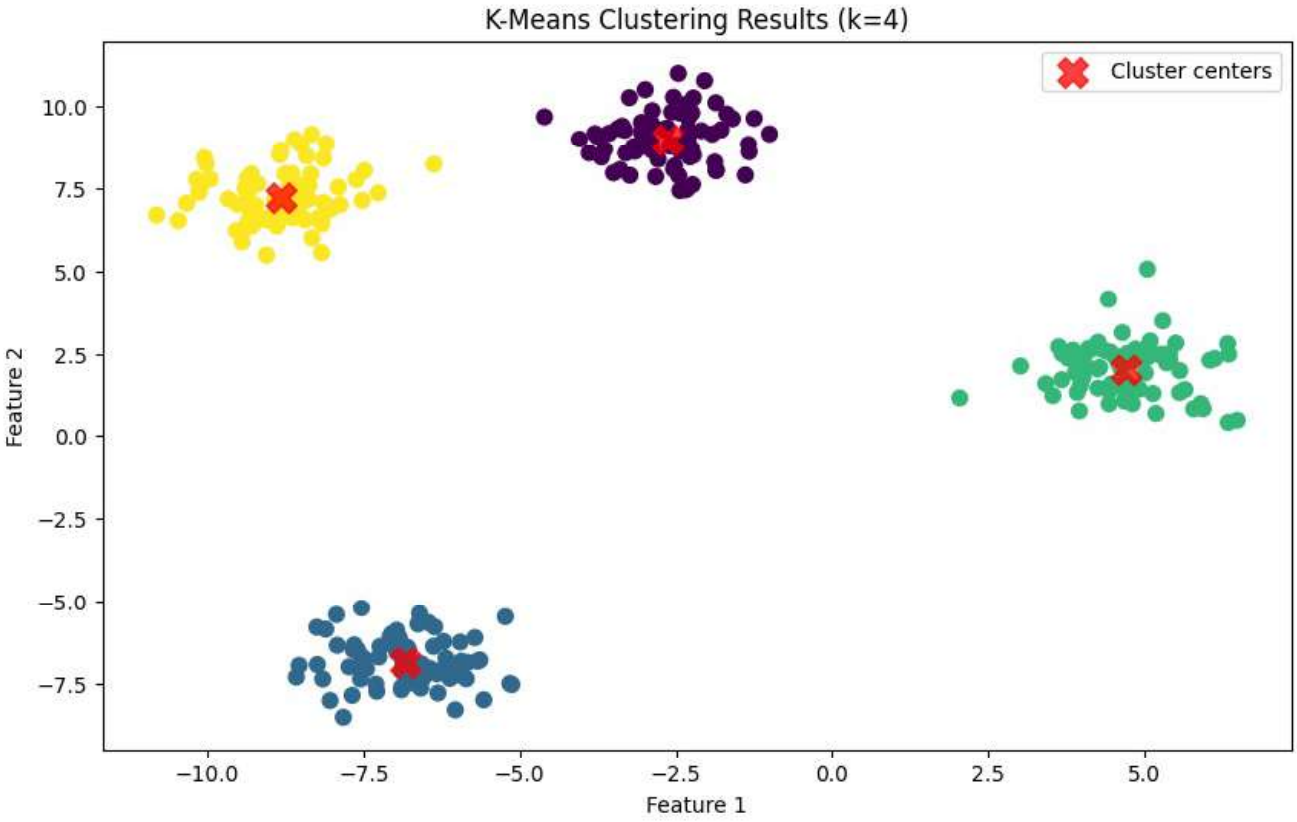
Your Task: Run K-Means with `n_clusters=4` and create a scatter plot showing the data points colored by their assigned cluster.

In [3]:
```python
# 1. Create and fit a KMeans model with n_clusters=4
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
kmeans.fit(X)

# 2. Get the cluster assignments for each data point
labels = kmeans.labels_

# 3. Get the coordinates of the cluster centers
centers = kmeans.cluster_centers_

# 4. Create a scatter plot of the data, colored by the assigned labels
plt.figure(figsize=(10, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap='viridis')
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.75, marker='X', label='Cluster centers')
plt.title('K-Means Clustering Results (k=4)')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()
```



Part 3: Principal Component Analysis (PCA)

Concept: PCA is a dimensionality reduction technique. It transforms the data into a new coordinate system of **principal components**. These components are orthogonal (uncorrelated) and are ordered by the amount of variance in the data they explain. By keeping only the first few principal components, we can reduce the number of dimensions in our data while losing as little information as possible.

Use Cases:

- **Visualization:** Reducing high-dimensional data to 2 or 3 dimensions so we can plot it.
- **Performance:** Speeding up model training by using fewer features.

Problem: We will use the Iris dataset, which has 4 features, and reduce it to 2 features so we can visualize it.

Task 3: Apply PCA to the Iris Dataset

Your Task: Use `PCA` from `sklearn.decomposition` to reduce the 4-dimensional Iris dataset to 2 principal components and visualize the result.

In [4]:
```
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris
import pandas as pd
import seaborn as sns

# Load the data
iris = load_iris()
X_iris = iris.data
y_iris = iris.target

# 1. Create a PCA instance to reduce to 2 components
pca = PCA(n_components=2)

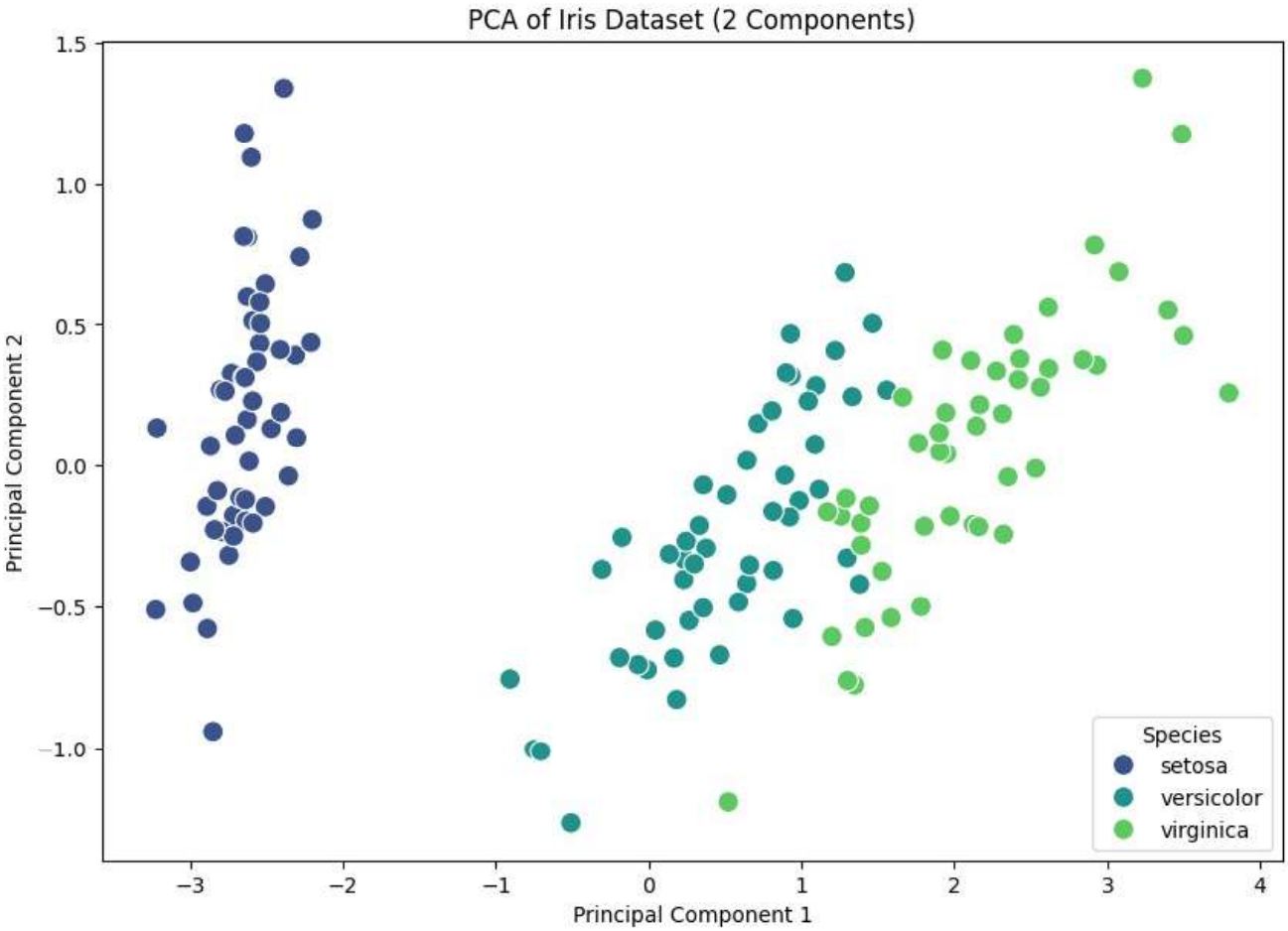
# 2. Fit PCA to the data and transform it
X_pca = pca.fit_transform(X_iris)

# 3. Print the explained variance ratio
print(f"Explained variance by component: {pca.explained_variance_ratio_}")
print(f"Total variance explained by 2 components: {pca.explained_variance_ratio_.sum():.2%}")

# 4. Create a DataFrame with the PCA results for plotting
pca_df = pd.DataFrame(data=X_pca, columns=["Principal Component 1", "Principal Component 2"])
pca_df["Species"] = [iris.target_names[i] for i in y_iris]

# 5. Plot the results
plt.figure(figsize=(10, 7))
sns.scatterplot(x="Principal Component 1", y="Principal Component 2", hue="Species", data=pca_df,
palette='viridis', s=100)
plt.title('PCA of Iris Dataset (2 Components)')
plt.show()
```

```
Explained variance by component: [0.92461872 0.05306648]
Total variance explained by 2 components: 97.77%
```


PCA of Iris Dataset (2 Components)
Principal Component 2
Principal Component 1
1.5
1.0
0.5
0.0
−0.5
−1.0
−3
−2
−1
0
1
2
3
4
Species
setosa
versicolor
virginica

Knowledge Check – Answers

1. What is the fundamental difference between supervised and unsupervised learning?

The fundamental difference is whether the training data comes with labels, meaning the “correct answers” you want the model to learn. Supervised learning trains on input-output pairs, while unsupervised learning trains on inputs alone, with no labels.

Supervised learning uses labeled data, the dataset includes input features (x) and a known target or correct outcome (y). The model learns to predict the target from the features. Unsupervised learning uses data without a target label; it looks for patterns, structure, or groups within the features.

Supervised learning has a clear objective: minimize the gap between predictions and labels. You can measure success directly with metrics like accuracy or error. Unsupervised learning optimizes something internal to the data, such as how tight the clusters are or how well the data can be reconstructed. Evaluation is often harder and more subjective, because there is no ground truth to compare against.

2. In the K-Means elbow plot, why don't we just choose the largest possible `k` to get the lowest inertia?

Inertia generally decreases as more clusters are added, hence choosing the largest k can create many small, less useful groups and over-segment the data. The goal is choosing a k that balances compact clusters with simplicity. The elbow is the point after which adding clusters produces only a relatively small improvement.

The core idea is that choosing k is a model selection problem. Balancing fit against simplicity, and any criterion that rewards fit alone will always push you toward more complexity than the data supports.

3. In the PCA results, the first two principal components explained over 95% of the variance. What does this tell you about the original 4-dimensional Iris dataset?

The first two principal components retain most of the variation in the original four features. Therefore, the Iris data can be represented in two dimensions for visualization with relatively little information loss. In this dataset, the two components explain approximately 97.77% of the total variance.

ITAI 1371 ? Module 10

Individual Student Submission

Yilin Leng

Unsupervised Learning Lab

Module 10 Lab - Unsupervised Learning**Objective:** To understand and apply two common **unsupervised learning** techniques: **K-Means Clustering** for finding groups in data and **Principal Component Analysis (PCA)** for dimensionality reduction.**In this lab, you will write the code to perform clustering and PCA, and you will interpret the results.**

Part 1: Supervised vs. Unsupervised LearningSo far, every model we have built has been a **supervised learning** model. This means we had **labeled data**—we had both the features (X) and the correct answer, or target (y). The goal was to learn a mapping from X to y.In **unsupervised learning**, we only have the features (X). There is **no target variable**. The goal is to discover interesting structures or patterns hidden in the data on its own.Two major types of unsupervised learning are:* **Clustering:** Automatically grouping similar data points together.* **Dimensionality Reduction:** Compressing the data by reducing the number of features while trying to preserve as much information as possible.

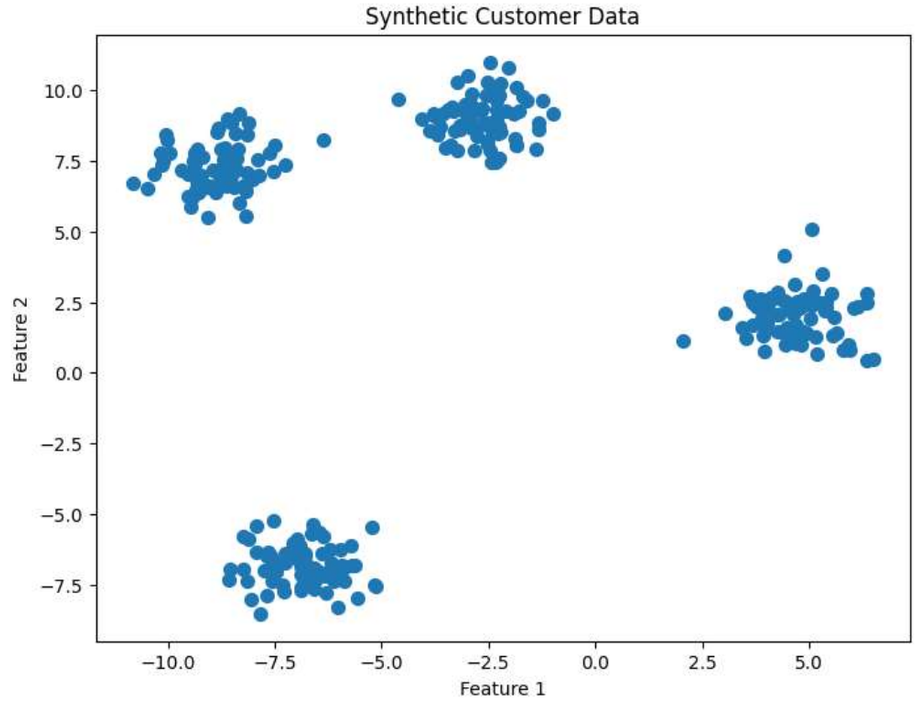
Part 2: K-Means Clustering**Concept:** K-Means is an algorithm that finds a pre-defined number of clusters (`k`) in a dataset. It works by:1. Randomly placing `k` centroids (the center of a cluster) in the data.2. **Assignment Step:** Assigning each data point to its nearest centroid.3. **Update Step:** Moving each centroid to the average position of all the points assigned to it.4. Repeating steps 2 and 3 until the centroids stop moving.**Problem:** We will use a synthetic dataset of customer spending habits to segment them into groups.

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

X, y = make_blobs(n_samples=300, centers=4, cluster_std=0.8, random_state=42)

plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], s=50)
plt.title("Synthetic Customer Data")
plt.xlabel("Feature 1")
```

```
plt.ylabel("Feature 2")
plt.show()
```



Task 1: Find the Optimal Number of Clusters (The Elbow Method)**Concept:** How do we know what k should be? The **Elbow Method** helps us decide. We run K-Means for a range of k values and for each k, we calculate the **inertia** (the sum of squared distances of samples to their closest cluster center). We then plot the inertia for each k. The "elbow" of the curve—the point where the inertia starts to decrease much more slowly—is a good estimate for the optimal k.**Your Task:** Calculate and plot the inertia for k values from 1 to 10.

```
from sklearn.cluster import KMeans

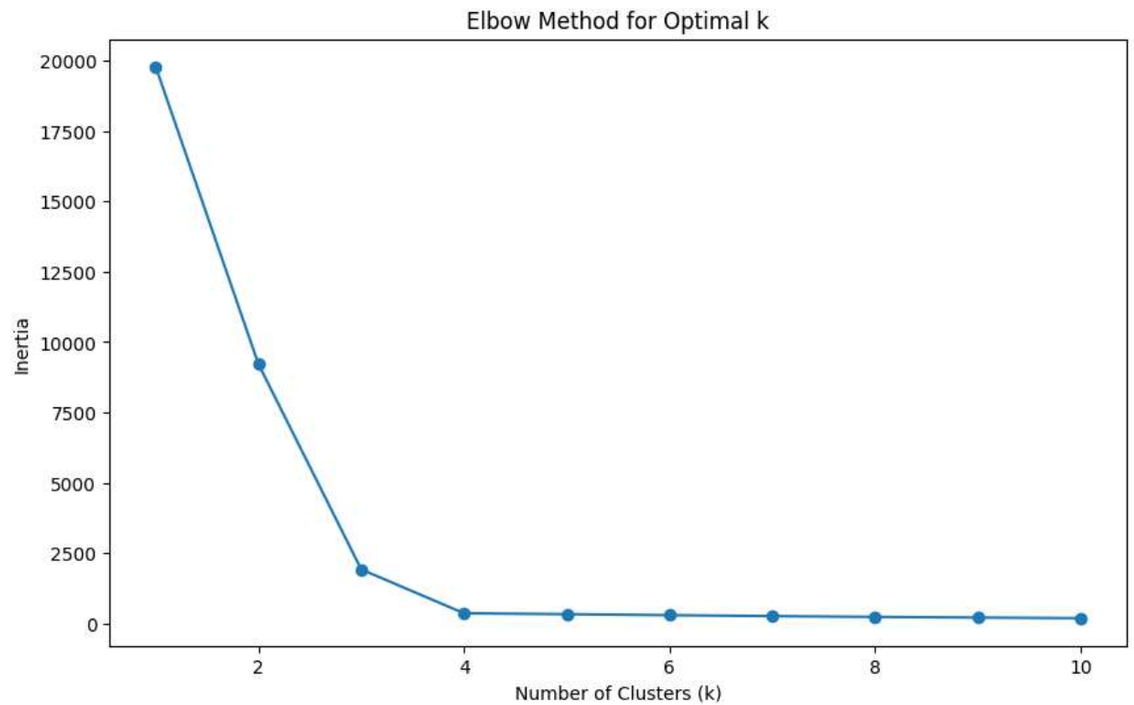
inertia = []

for k in range(1, 11):

    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X)
```

```
    inertia.append(kmeans.inertia_)

plt.figure(figsize=(10, 6))
plt.plot(range(1, 11), inertia, marker="o")
plt.title("Elbow Method for Optimal k")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Inertia")
plt.show()
```



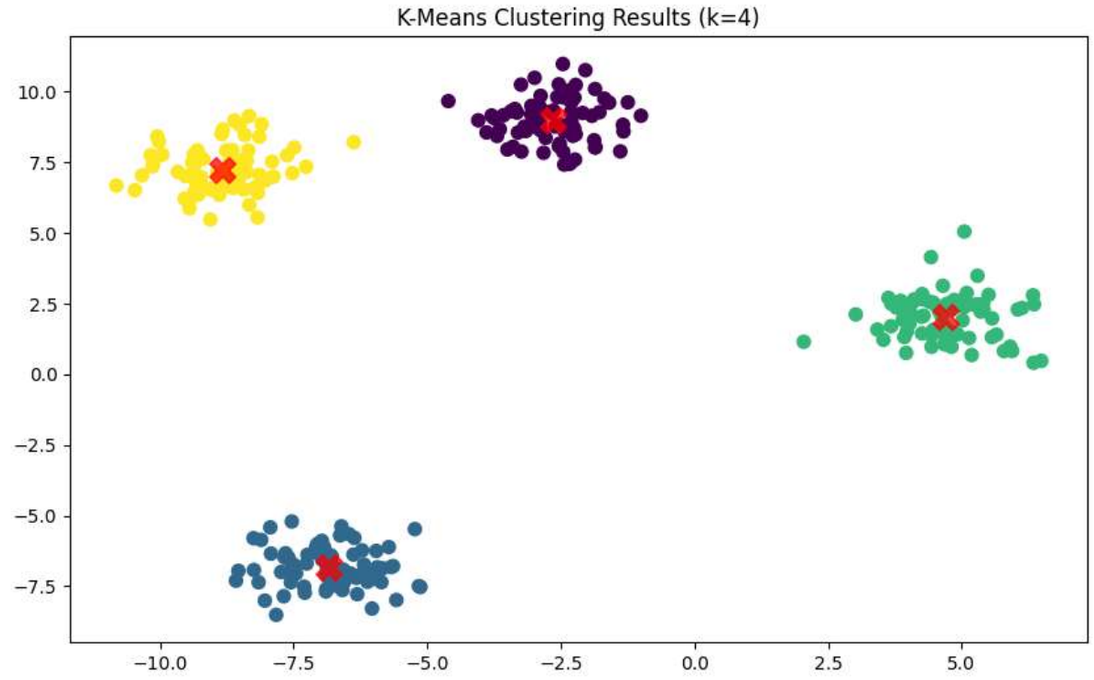
Task 2: Perform K-Means Clustering and VisualizeFrom the elbow plot, it looks like `k=4` is the optimal number of clusters.**Your Task:** Run K-Means with `n_clusters=4` and create a scatter plot showing the data points colored by their assigned cluster.

```
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
kmeans.fit(X)

labels = kmeans.labels_

centers = kmeans.cluster_centers_
```

```
plt.figure(figsize=(10, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap="viridis")
plt.scatter(
    centers[:, 0], centers[:, 1], c="red", s=200, alpha=0.75, marker="X"
)
plt.title("K-Means Clustering Results (k=4)")
plt.show()
```



Part 3: Principal Component Analysis (PCA)**Concept:** PCA is a dimensionality reduction technique. It transforms the data into a new coordinate system of **principal components**. These components are orthogonal (uncorrelated) and are ordered by the amount of variance in the data they explain. By keeping only the first few principal components, we can reduce the number of dimensions in our data while losing as little information as possible.**Use Cases:*** **Visualization:** Reducing high-dimensional data to 2 or 3 dimensions so we can plot it.* **Performance:** Speeding up model training by using fewer features.**Problem:** We will use the Iris dataset, which has 4 features, and reduce it to 2 features so we can visualize it.

Task 3: Apply PCA to the Iris Dataset**Your Task:** Use `PCA` from `sklearn.decomposition` to reduce the 4-dimensional Iris dataset to 2 principal components and visualize the result.

```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA

iris = load_iris()
X_iris = iris.data
y_iris = iris.target

pca = PCA(n_components=2)

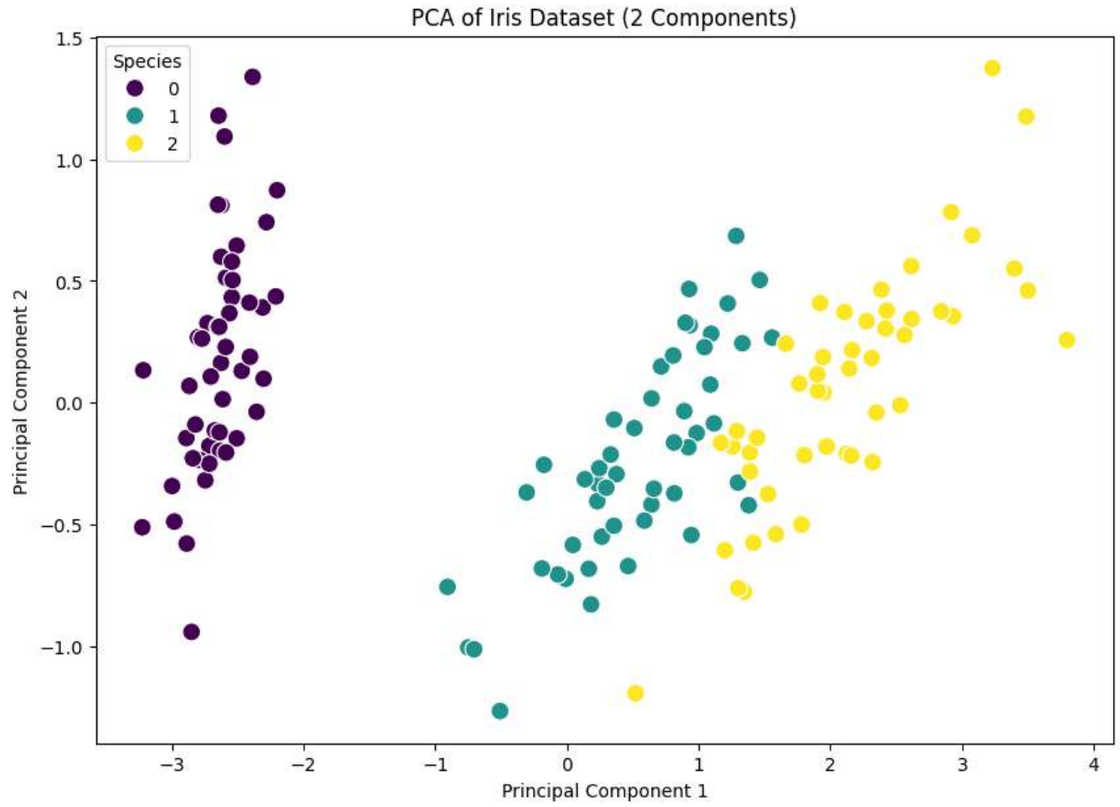
X_pca = pca.fit_transform(X_iris)

print(f"Explained variance by component: {pca.explained_variance_ratio_}")
print(
    f"Total variance explained by 2 components: {pca.explained_variance_ratio_.sum():.2%}"
)

pca_df = pd.DataFrame(
    data=X_pca, columns=["Principal Component 1", "Principal Component 2"]
)
pca_df["Species"] = y_iris

plt.figure(figsize=(10, 7))
sns.scatterplot(
    x="Principal Component 1",
    y="Principal Component 2",
    hue="Species",
    data=pca_df,
    palette="viridis",
    s=100,
)
plt.title("PCA of Iris Dataset (2 Components)")
plt.show()
```

```
Explained variance by component: [0.92461872 0.05306648]
Total variance explained by 2 components: 97.77%
```



📝 Knowledge Check**Instructions:** Answer the following questions in this markdown cell.1. **What is the fundamental difference between supervised and unsupervised learning?**2. **In the K-Means elbow plot, why don't we just choose the largest possible `k` to get the lowest inertia?**3. **In the PCA results, the first two principal components explained over 95% of the variance. What does this tell you about the original 4-dimensional Iris dataset?**

1. What is the fundamental difference between supervised and unsupervised learning?

   Supervised Learning: Uses labeled data, which includes features (X) and known answers (y). The model learns from these examples to make predictions. Common tasks include classification and regression. Unsupervised Learning: Uses unlabeled data, which only

contains features (X). The model tries to find hidden patterns or groups in the data. Common tasks include clustering and dimensionality reduction.

2. In the K Means elbow plot, why don't we just choose the largest possible k to get the lowest inertia?

   A larger k usually leads to lower inertia because each cluster contains fewer data points. However, choosing too many clusters can make the model too complex. In the extreme case, each data point becomes its own cluster, which defeats the purpose of clustering. The elbow method helps us find a good balance between low inertia and a reasonable number of clusters.

3. In the PCA results, the first two principal components explained over 95% of the variance. What does this tell you about the original 4 dimensional Iris dataset?

   This means that most of the important information in the original four features can be represented by just two principal components. Although the Iris dataset has four dimensions, the first two principal components explain more than 95% of the total variance. This shows that PCA can reduce the number of dimensions while keeping most of the important information. It also makes the data easier to visualize and analyze.